3 Sesión 3: Ranking.

3.1 Bloque A: Conjunto de datos de ranking.

En este bloque se muestra como almacenar la información de ejemplos cuando la categoría es un ranking. La estructura de los atributos(X) es la misma, pero lo que cambia es la categoría (Y).

La información que hay en la categoría es la que permite definir un ranking, posiblemente parcial, entre un conjunto de ejemplos. Este conjunto de ejemplos para los que hay un ranking se llama lote. Para ello se utilizan dos valores, uno entero que identifica el lote y otro numérico que identifica el ranking de cada ejemplo el lote. En el caso especial de que solo haya un lote se puede indicar con solo un valor, el del ranking.

A continuación, se muestra gráficamente la información del ejemplo 2.

	X	Y	
Restaurante	**Atributos**	**[V, L]**	**Lote**
R2	[0.5,-1]	[2, 1]	1
R1	[9.4, 5]	[1, 1]	
R2	[0.5,-1]	[3, 2]	2
R4	[7.5, 2.3]	[2, 2]	
R3	[3.5, 0]	[2, 2]	
R1	[9.4, 5]	[1, 2]	
R2	[0.5,-1]	[4, 3]	3
R3	[3.5, 0]	[3, 3]	
R4	[7.5, 2.3]	[2, 3]	
R1	[9.4, 5]	[1, 3]	

V: Valor de ranking
L: Lote

Realiza el ejercicio 1.

3.2 Bloque B: Aprendizaje a partir de ranking bipartito en un solo lote.

En este bloque se ha creado un conjunto bipartito de entrenamiento (`XTr,YTr`) y otro de test (`XTe,YTe`) que so originados a partir del mismo modelo definido por el vector `w`.

T1_S3_B_RankingBipartito.py

```
nAtt=10      # Atributos

RG=random.Random() # Generador de aletoriedad
RG.seed(1)
w=list(RG.gauss(0,1) for i in range(nAtt)) # Pesos(w) de la función lineal
nEjeTr=100  # Ejemplos de entrenamiento
nEjeTe=100  # Ejemplos de test
ruido=1     # Ruido. 0 no hay ruido, cuanto mayor, más ruido

[XTr,YTr]=genXY(RG,w,nEjeTr,ruido)
[XTe,YTe]=genXY(RG,w,nEjeTe,ruido)
```

También se presenta la clase `linearRank` que realiza un aprendizaje de un ranking utilizando un clasificador lineal.

```
lRank=linearRank()
Model=lRank.fit(XTr,YTr)
```

El modelo obtenido se puede obtener mediante: `Model.LinealClass.coef_[0]`

```
In [10]: Model.LinealClass.coef_[0]
Out[10]:
array([ 0.02442562, -0.29055593,  0.1482728 ,  0.01320995, -0.09095237,
       -0.00437392, -0.01968563, -0.03689037, -0.18774424,  0.05721073])
```

Se puede obtener el AUC usando la medida `roc_auc_score`. El AUC del conjunto de test es aprox. 92,93%

```
Eval=Model.predict(XTe)
AUC=roc_auc_score(YTe,Eval)
```

¿Qué pasa si aumenta el ruido? ¿Aumentará o disminuirá el AUC? ¿Hasta donde?

Realiza el ejercicio 1.

3.3 Bloque C: Aprendizaje de ranking multipartito de un solo lote.

En este bloque se toma el conjunto de datos `qsar_aquatic_toxicity`, que tiene una categoría con 11 valores enteros en [0;10]. Se trata como un problema de ranking y se usa para aprender `linearRank`.

Hay dos novedades, como la categoría tiene varios valores (hay 11 no solo 2) se usa *C-Index* y para estimar el error se usa validación cruzada.

```
Ranker=linearRank(verbose=1)
cv=cross_validate(Ranker,X,Y,scoring=make_scorer(CIndex),cv=3)
```

Fíjate en la traza del primer *fold*. A partir de un lote de longitud 66 se generan 1781 pares que dan lugar a 3562 ejemplos. Cuidado! Conjunto de datos que parecen pequeños podrían generar gran cantidad de ejemplos. Si el ranking es total, para m objetos se tienen $m^2 - m$ ejemplos.

Realiza el ejercicio 1.

3.4 Bloque D: Aprendizaje de ranking con varios lotes.

La primera parte es similar al bloque A. Termina con la predicción en P de los ejemplos de test. Esta predicción es un valor numérico que sirve para comparar estos restaurantes (objetos).

```
Objeto           | Valor
RN3 valor ranking:+0.4571
RN2 valor ranking:-0.8980
RN4 valor ranking:-1.1583
RN5 valor ranking:-2.1605
RN1 valor ranking:-2.7447
```

En la siguiente parte se proporcionan valores de críticos, éstos se consideran los verdaderos y se quiere calcular una medida de calidad del ranking anterior con respecto a estos valores verdaderos.

Cada crítico valora varios restaurantes, pero las valoraciones no se pueden comparar entre si. El crítico 1 valora en [0,5], el crítico 2 en [0,10] mientras que el crítico 2 en [0,100]. Los lotes serán los que indiquen que solo se tienen que establecer preferencias entre los restaurantes que haya evaluado cada crítico.

La información de los críticos es:

```
Valoraciones=[
    #RN1 ,RN2 ,RN3 ,RN4 ,RN5  <- Restaurante (Objeto)
    [3   ,None,None,4   ,2],  # Crítico 1
    [2   ,4   ,None,None,8],  # Crítico 2
    [None,27  ,53  ,45  ,27]] # Crítico 3
```

Crítico 1 → Lote 0
Crítico 2 → Lote 1
Crítico 3 → Lote 2

La información de las valoraciones de los expresadas en formar de par [valor, lote] y las valoraciones del modelo se han generado en `YTe` y en `PTe` donde al crítico *i* se le asigna el lote (*i-1*).

```
YTe=[[3,0],[4,0],[2,0],[2,1],[4,1],[8,1],[27,2],[53,2],[45,2],[27,2]]
XTe=[RN1  ,RN4  ,RN5  ,RN1  ,RN2  ,RN5  ,RN2   ,RN3   ,RN4   ,RN5   ]
```

Se evalúa de manera tradicional:

```
PTe=Ranker.predict(XTe)
```

Fíjate que se imprime la evaluación:

```
RN1 valor ranking:-2.7447
RN4 valor ranking:-1.1583
RN5 valor ranking:-2.1606
RN1 valor ranking:-2.7447
RN2 valor ranking:-0.8980
RN5 valor ranking:-2.1606
RN2 valor ranking:-0.8980
RN3 valor ranking:+0.4572
RN4 valor ranking:-1.1583
RN5 valor ranking:-2.1606
```

Estos valores coinciden con el valor de cada objeto de la primera pare de este bloque ya que el modelo es el mismo y los objetos a evaluar son, también, los mismos.

Fíjate en el valor del mejor (RN3) y el peor (RN1)

La llamada a `multiBatchCIndex` calcula el *C-Index* como media de los *C-Index* de cada lote.

3.4.1 Juntar lotes de distintos datasets.

Cada dataset tiene varios lotes que tienen que numerarse de manera diferente, pero no tienen que tener números diferentes a los de otros datasets.

Cuando se trata de juntar varios datasets, los números de lotes diferentes no pueden coincidir por que si no serían el mismo lote.

La función `aggregateBatchDataSets` en `Rank` realiza este proceso de manera adecuada.

Ejemplo de uso:

```
YDS1=[[1,0],[3,0],[1,1],[2,1]]
YDS2=[[2,0],[5,0],[2,0],[2,1],[4,1],[4,1],[1,3],[2,3],[2,3]]
YDS3=[[1,0],[1,0],[2,0]]
newYDS=aggregateBatchDataSets(YDS1,YDS2,YDS3)
print('NewYDS:')
for vb in newYDS:
    print(vb)
```

```
NewDS:
[1, 0]
[3, 0]
[1, 1]
[2, 1]
[2, 2]
[5, 2]
[2, 2]
[2, 3]
[4, 3]
[4, 3]
[1, 4]
[2, 4]
[2, 4]
[1, 5]
[1, 5]
[2, 5]
```

Realiza el ejercicio 1.

3.5 Bloque E: Estimación del error con varios lotes.

3.5.1 Se supone que todos los objetos son diferentes.

Cuando hay varios lotes se querría estimar el error para lotes nuevos. Estos lotes nuevos siempre tendrían ejemplos diferentes, pues se consideren todos diferentes entre si.

La manera de conseguir esta estimación sería mediante una validación cruzada en la que en entrenamiento haya unos lotes y en test otros diferentes.

La validación cruzada tiene la opción de agrupar por grupos, de manera que serán esos grupos los que distribuyan (enteros) por cada *fold*. En este caso si se indica que cada grupo es un lote, se conseguirá que los pares de entrenamiento y test sean de lotes diferentes.

```
X=[R2    ,R1    ,R2    ,R4    ,R3    ,R1    ,R2    ,R3    ,R4    ,R1]
Y=[[2,1] ,[1,1] ,[3,2] ,[2,2] ,[2,2] ,[1,2] ,[4,3] ,[3,3] ,[2,3] ,[1,3]]
```

```
ranker=linearRank()

# Los grupos son los lotes
groups=[]
for [value,idLote] in Y:
    groups.append(idLote)

# Se ha de utilizar una validación cruzada donde se indica en groups los lotes
#y se utiliza como cv un objeto de la clase GroupKFold
CVscores=cross_validate(ranker,X,Y,scoring=make_scorer(multiBatchCIndex),
                        groups=groups,cv=GroupKFold(n_splits=3))
```

```
# No puede haber más folds que lotes
```

```
In [24]: Y
Out[24]:
[[2, 1],
 [1, 1],
 [3, 2],
 [2, 2],
 [2, 2],
 [1, 2],
 [4, 3],
 [3, 3],
 [2, 3],
 [1, 3]]

In [25]: groups
Out[25]: [1, 1, 2, 2, 2, 2, 3, 3, 3, 3]
```